

Laxmi Charitable Trust's
Sheth L.U.J College of Arts & Sir M.V. College of Science and Commerce
Department of Information Technology (B.Sc.I.T Semester IV)
 Artificial Intelligence

Practical – II

Roll No. : T014	Name: Parth R. Kunkunkar
Class : TYIT	Date/Time of Submission : 27/06/2026

Aim: Problem Solving by Searching (Uninformed Search)

1). Given an initial configuration of the 8-puzzle and a goal configuration, write a Python program to find the shortest sequence of moves to reach the goal state using Breadth-First Search (BFS).

CODE:

```
#Q1
from collections import deque

# Function to find the position of the blank tile (0)
def find_blank(state):
    for i in range(3):
        for j in range(3):
            if state[i][j] == 0:
                return i, j

# Function to generate all possible moves
def get_neighbors(state):
    neighbors = []
    row, col = find_blank(state)

    moves = [(-1, 0), (1, 0), (0, -1), (0, 1)] # Up, Down, Left, Right

    for dr, dc in moves:
        new_row = row + dr
        new_col = col + dc

        if 0 <= new_row < 3 and 0 <= new_col < 3:
            # Make a copy of the puzzle
            new_state = [list(r) for r in state]

            # Swap blank with adjacent tile
            new_state[row][col], new_state[new_row][new_col] = \
                new_state[new_row][new_col], new_state[row][col]

            neighbors.append(tuple(tuple(r) for r in new_state))

    return neighbors

# BFS function
def bfs(start, goal):
    queue = deque()
```

```
queue.append((start, []))

visited = set()
visited.add(start)

while queue:
    current, path = queue.popleft()

    if current == goal:
        return path + [current]

    for neighbor in get_neighbors(current):
        if neighbor not in visited:
            visited.add(neighbor)
            queue.append((neighbor, path + [current]))

return None

# Function to print the puzzle
def print_state(state):
    for row in state:
        print(row)
    print()

# Initial state
start = (
    (1, 2, 3),
    (4, 0, 6),
    (7, 5, 8)
)

# Goal state
goal = (
    (1, 2, 3),
    (4, 5, 6),
    (7, 8, 0)
)

# Find solution
solution = bfs(start, goal)

if solution:
    print("Solution found in", len(solution) - 1, "moves.\n")

    for step in solution:
        print_state(step)
else:
    print("No solution found.")
```

OUTPUT:

```
Solution found in 2 moves.
```

```
(1, 2, 3)
(4, 0, 6)
(7, 5, 8)
```

```
(1, 2, 3)
(4, 5, 6)
(7, 0, 8)
```

```
(1, 2, 3)
(4, 5, 6)
(7, 8, 0)
```

2). Given two water jugs of 4 litres and 3 litres capacity, write a Python program to obtain exactly 2 litres in one jug using Depth-First Search (DFS). [Vary capacity of jugs]

CODE:

#Q2

DFS for Water Jug Problem

```
def dfs(jug1, jug2, visited):
    # Goal: Get exactly 2 litres in any jug
    if jug1 == 2 or jug2 == 2:
        print(f'Goal Reached: ({jug1}, {jug2})')
        return True

    if (jug1, jug2) in visited:
        return False

    visited.add((jug1, jug2))

    print((jug1, jug2))

    # Possible operations
    next_states = [
        (4, jug2),          # Filling Jug1
        (jug1, 3),          # Filling Jug2
        (0, jug2),          # Emptying Jug1
        (jug1, 0),          # Emptying Jug2
        (jug1 - min(jug1, 3 - jug2), jug2 + min(jug1, 3 - jug2)), # Jug1 -> Jug2
        (jug1 + min(jug2, 4 - jug1), jug2 - min(jug2, 4 - jug1)) # Jug2 -> Jug1
    ]

    for state in next_states:
        if dfs(state[0], state[1], visited):
            return True

    return False

visited = set()
```

```
print("Steps:")
dfs(0, 0, visited)
```

OUTPUT:

```
... Steps:
    (0, 0)
    (4, 0)
    (4, 3)
    (0, 3)
    (3, 0)
    (3, 3)
    Goal Reached: (4, 2)

... True
```

3). Given a weighted graph representing cities and distances between them, write a Python program to find the least-cost path from Arad to Bucharest using Uniform Cost Search (UCS). [Provide any other weighted graph for applying UCS]

CODE:

```
import heapq

# Graph of Indian cities with distances
graph = {
    "Mumbai": [("Pune", 150), ("Nashik", 170)],
    "Pune": [("Mumbai", 150), ("Kolhapur", 230)],
    "Nashik": [("Mumbai", 170), ("Aurangabad", 200)],
    "Kolhapur": [("Pune", 230), ("Solapur", 300)],
    "Aurangabad": [("Nashik", 200), ("Solapur", 250)],
    "Solapur": [("Kolhapur", 300), ("Aurangabad", 250), ("Hyderabad", 300)],
    "Hyderabad": [("Solapur", 300)]
}

def ucs(start, goal):
    queue = []
    heapq.heappush(queue, (0, start, [start]))
    visited = set()

    while queue:
        cost, city, path = heapq.heappop(queue)

        if city == goal:
            print("Least Cost:", cost)
            print("Path:", " -> ".join(path))
            return

        if city not in visited:
            visited.add(city)

            for next_city, distance in graph[city]:
                if next_city not in visited:
                    heapq.heappush(queue, (cost + distance, next_city, path + [next_city]))

# Start and Goal
ucs("Mumbai", "Hyderabad")
```

OUTPUT:

```
Least Cost: 920
```

```
Path: Mumbai -> Nashik -> Aurangabad -> Solapur -> Hyderabad
```